yee-search

Searching for voting methods that resist burial: what is scored, on which configurations, and how the searches find methods

`search/` is a project of its own (its own `uv` lock and environment) that builds voting methods out of the blocks of `yeelab.build` and looks for those that stay close to the nearest candidate both when every voter is sincere and when 30 % of the voters bury. Section 1 describes the methods it can build, Section 2 how one method is scored, Section 3 the configurations of candidates it is scored on, Section 4 how the scoring is made fast, Section 5 and Section 6 the two searches, Section 9 how the methods found are chosen and checked, Section 10 how to run it all, and Section 11 the methods of the web UI's row "Found by search".

1. Methods as trees of blocks

A method is a tree of blocks of `yeelab.build` (docs/math.pdf, chapter on the voting methods): ballots (`Plurality`, `BordaCount`, `AntiPlurality`, `Score`), totals (`Tally` of the mean or the median, `Weakest`), pairwise comparisons (`Margins`, `ScoreComparisons`, `StrongestPaths`), methods (`Highest`, `Eliminate`, `Unbeaten`, `DescendingCoalitions`), sets of candidates (`Highest(..., n=2)`, `Smith`, the union `|` of two winners) and conditions (`A == B`, a measure `Top` or `Lead` against a threshold, `&`, `|`) for `Conditional(condition, when_true, when_false)`.

The grammar (`grammar.py`) types every slot of a tree, so that only blocks that fit go there: a **method** elects one candidate, **among** chooses candidates for another method to choose from, **if** is the condition of a Conditional (a condition, or a method that is true where it elects someone), a **measure** is `Top` or `Lead` of a tally of the mean, and so on. The constants are few on purpose: the score ballots `Score(2)` and `Score(6)` with power 1.5, the tie weights 0 and 1 of the score runoff, the thresholds 0.02, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.55, 0.6 and 0.7, two or three finalists, and Smith sets of a majority of 0.5, 0.55 or 0.6. D20 (two votes for the first two, `Highest(Tally(D21(...)))`) is a method of its own, a leaf: weak alone, it may tell where others are right inside a Conditional. `Undecided()` is left out: a method must try to elect someone everywhere. A tree is at most 12 deep and has at most 60 nodes.

2. Scoring a method

2.1. The configuration and its diagram

A configuration is a set of candidates in the unit square and the deviation d of the Beta voters around every point (the voter model of YeeLab with the spread rule `rms`). The method is run on the shares of the voters at every point of a grid of G cells per side, $(G+2)^2$ points with both walls: $G = 16$ for training, 24 for validation and 32 for the test (Section 9). Every point of the grid is drawn at the **geometric median** of its voters, as the web UI starts ("Point in diagram is $p = \arg\min_q E\|V - q\|$"): the winners are those of the voters of the grid, only placed elsewhere, and near the walls the diagram leaves a strip empty (docs/math.pdf, the geometric diagram).

2.2. Burial

The method is scored twice: with every voter sincere, and with 30 % of the voters burying at every point, as the web UI's "Burying voters" does. The frontrunners at a point are the two with the most first choices (`Burial`'s default poll), and a burying voter ranks the one of them they like less last and gives it 0 on a score ballot. FPTP and the two-round system cannot be moved so: burial changes neither the first choices nor the head-to-head of the two frontrunners.

2.3. The error

At a point x that elects the candidate w the loss is

$$\ell(x, w) = \frac{r}{r+s}, \quad r = \|x - w\| - \min_c \|x - c\|, \tag{1}$$

s the median distance between two candidates: 0 where the nearest candidate wins, close to 1 far from it. A point without a winner (a cycle, a draw) counts as won by the farthest candidate, so that a method cannot leave out the points it decides badly. The error of the method over an area A is relative to a candidate drawn at random at every point,

$$e_A = \frac{\sum_{x \in A} a_x \ell(x, w_x)}{\sum_{x \in A} a_x \frac{1}{C} \sum_c \ell(x, c)}, \tag{2}$$

a_x the area of the point. It is taken over the region around the candidates (the dashed outline of the web UI's "metrics region") and over the whole diagram, and

$$e = 0.8 e_{\text{region}} + 0.2 e_{\text{whole}}, \tag{3}$$

mostly where the candidates are, but so that a method that elects the wrong candidate far from all of them pays too. The web UI shows $\sqrt{e_A}$ of each area; the search does not take the root. For configuration k, e_k is this error with sincere voters and b_k with 30 % burying.

2.4. The objective

$$J = \operatorname*{trimmed\,mean}_k (0.25 e_k + 0.75 b_k) + 0.0001 \cdot \text{complexity}, \tag{4}$$

lower being better, the focus configurations (Section 3) each counting as 3 configurations and never trimmed. On them a method must bury no worse than `search_6`, a hard rule: with b_k the burial error and

$$\text{behind} = \sum_{k \in \text{ focus}} \max\left(0, \frac{b_k}{b_k}(\text{search_6}) - 1\right), \tag{5}$$

a method of behind > 0 gets $1 +$ behind added to J. It ranks below every method that keeps the rule, whose J is far below 1, and the others among themselves by how far behind they are, so that the genetic programming still has a slope to climb towards the rule. The report shows only the methods of behind $= 0$. Burial weighs three times as much as the sincere error: a method that only resists burial is FPTP, whose sincere error is large. The trimmed mean leaves out, for each method, the 5 % of the configurations it does best on and the 5 % it does worst on ($\lfloor 0.05K \rfloor$ of each end of K configurations), so that a few configurations do not decide the order; the sincere and the burial error reported with J are the means over the same kept configurations. A method that fails or takes too long has $J = \infty$.

2.5. Complexity

The complexity counts the blocks of a method, so that of two methods that err alike the simpler one wins. Let $T(m)$ be the tree of blocks of m and $D(m)$ the set of its *distinct* subtrees: two blocks are the same if they have the same type, parameters and children (the same `repr`), so that a block that recurs is one element of $D(m)$. Let F be the free types `Tally`, `Weakest` and `Union`, and $C(m) \subset D(m)$ the distinct Conditionals. The depth $\delta(c)$ of a Conditional c is the number of Conditionals that enclose it in $T(m)$, in a branch or in a condition, found where the tree is first entered. Let $\alpha(m)$ be the number of `among=` set in $T(m)$, each occurrence: a subtree that recurs counts its `among=` as often as it recurs. Then

$$\text{complexity}(m) = \underbrace{|\{b \in D(m) : \text{type}(b) \notin F\}|}_{\text{blocks paid for}} + 2 \sum_{c \in C(m)} \delta(c) + \alpha(m). \tag{6}$$

The cost in J is 0.0001 times this.

- **Distinct blocks.** A method in a condition and in a branch, or the score ballot that four blocks read, is paid for once, as `methods.py` writes it once by name.
- **Free types.** Every ballot needs a `Tally` to give a winner, every pairwise comparison a `Weakest` or an `Unbeaten`, and a `Union` (`A | B`) only lists two finalists, so none is paid for: only what is inside them. A union costs the blocks of A and B and nothing for the `|`.

- **Nesting.** A Conditional pays 2 more for each Conditional around it. A chain $\text{Conditional}(c_1, A, \text{Conditional}(c_2, B, \text{Conditional}(c_3, C, D)))$ has depths $0, 1, 2$ and pays $2(0+1+2) = 6$ beyond its blocks; a balanced one, a root with a Conditional in each branch, has depths $0, 1, 1$ and pays $2(0+1+1) = 4$. One Conditional of a combined condition, $\text{Conditional}((x == y) \mid (y \neq z), A, B)$, has depth 0 and pays nothing beyond its blocks: a `CombinedCondition` and two `WinnerComparisons` are blocks, but no Conditional is nested.
- Parameters (`n=2`, `how="mean"`, a threshold, a tie weight) are free.
- **`among=`.** It is a field of `Highest`, `Eliminate` and `Unbeaten` like any other, so its value is a subtree counted by the same rule: `Highest(Tally(Plurality()), n=2)` costs two blocks (`Highest` and `Plurality`, the `Tally` being free), and `among=A | B` the blocks of A and B together, those they share counted once. Every `among=` costs 1 more, and this each time it occurs, not once: with distinct blocks alone a subtree copied into another `among=` was nearly free, and the genetic programming grew chains of `among=` that repeat the same finalists (5 to 7 of them in a third of its best methods). A runoff (STAR, the two-round system) pays 1 for its finalists.

Examples. STAR, `Unbeaten(ScoreComparisons(Score(6)), among=Highest(Tally(Score(6)), n=2))`, has the distinct blocks `Unbeaten`, `ScoreComparisons`, `Score(6)` (read twice, paid once), `Highest` and `Tally`; without the free `Tally` that is 4, and its one `among=` makes complexity $= 5$. `search_6` has 20 distinct blocks: 5 different `Highest`, 5 different `Tally`, and one each of `Conditional`, `WinnerComparison`, `Unbeaten`, `ScoreComparisons`, `Union`, `Eliminate`, `BordaCount`, `Plurality`, `Score(6)` and `D21`. Without the 5 `Tally` and the `Union` it is $20 - 6 = 14$; its only Conditional is the root, $\delta = 0$, and it sets 3 `among=`, so complexity $= 17$ and the cost 0.0017. `search_5` has a Conditional in the else-branch of its root, $\delta = 1$, which adds 2 beyond its blocks, and 2 `among=`, giving 22.

method	blocks	distinct blocks	complexity
`fptp`	3	3	2
`star`	6	5	5
`star_two_round`	13	12	11
`search_1`	10	9	8
`search_5`	32	24	22
`search_6`	27	20	17
`adhoc`	47	18	23
`con_fptp`	40	18	24

Table 1: Blocks, distinct blocks and complexity of some methods of `methods.py`: the chains `adhoc` and `con_fptp` pay for their nesting.

3. The configurations

3.1. Random configurations and hand-made families

A random configuration has 3, 4, 5 or 6 candidates drawn uniformly in $[0.1, 0.9]^2$, each two at least 0.08 apart, and a deviation of 0.1, 0.2 or 0.3. The hand-made families are the hard cases: *dark horse* (two frontrunners close to the centre and a weak third far away, who can win where the voters of each frontrunner bury the other), *dark horse four* (one more candidate on the other side), *center squeeze* (a Condorcet winner at the centre with few first choices, three around it), *clones* (two nearly equal candidates and two others), *collinear* (candidates on a line), *near walls* (candidates at the walls and corners), *triangle* (an equilateral triangle, where burial easily makes a cycle) and *outlier* (three close together, one far away). A family is a function of its parameters, so that the sets hold other instances of it: turned about the centre, spread wider or narrower, with other voters.

3.2. The sets

set	made of	grid	configurations
train	12 random (seed 1); the families at 0°, scale 1, deviation 0.2; the two focus configurations	16; the focus ones 64	20
wide	train but the focus configurations; 36 random (seed 6); the families at 200°, scale 1.05, deviation 0.12	16	58
validation	20 random (seed 2); the families at 90°, scale 0.9, deviation 0.15	24	25
test	200 random (seed 3); the families at 30°, 135° and 250° (scales 1.1, 0.8, 1, deviations 0.1, 0.25, 0.3); out of the training distribution: 20 of 7 or 8 candidates (seed 4), 20 of deviation 0.05 or 0.4 (seed 5)	32	230

Table 2: The sets of configurations, which share none. GP trains on `train`, the search of Conditionals on `wide`.

Focus configurations. Two configurations of the web UI (deviation 0.2) that the methods found by 2026-10-29 did not get right are in the training set on a grid of 64, each counting as 3 configurations in J and never trimmed (`configs.focus_configs`): four candidates nearly on a diagonal, the two in the middle close together, and three candidates around a fourth. They are in no other set: the search of Conditionals keeps one grid for all its maps.

A configuration is left out of every set where a candidate is nearer than 0.1 to a wall, or nearer than 0.05 to the strip the geometric diagram leaves empty along the walls: such a candidate is drawn at the very edge of the web UI's diagram or beyond it. The strip is wider for wider voters:

deviation	0.05	0.1	0.12	0.15	0.2	0.25	0.3	0.4
empty strip	0.01	0.018	0.021	0.027	0.038	0.052	0.07	0.124

Table 3: How far from a wall, at most, the geometric diagram starts.

The rule leaves out 2 configurations of the training set, 6 of `wide`, 3 of the validation set and 34 of the test set: every instance of *near walls*, two of *outlier* in the test, and random ones with a candidate near a wall and wide voters.

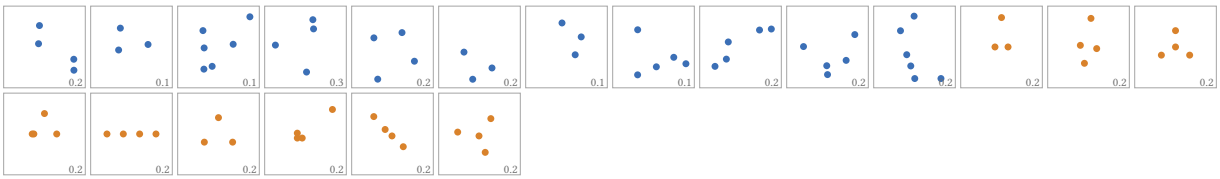


Figure 1: The train set: the candidates of every configuration in the unit square and its deviation; random ●, hand-made ●, out of the training distribution ●.

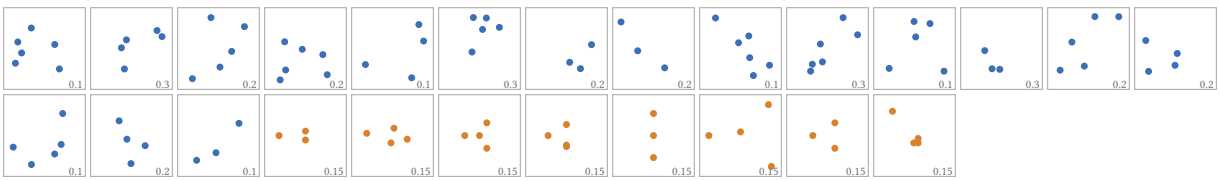


Figure 2: The validation set: the candidates of every configuration in the unit square and its deviation; random ●, hand-made ●, out of the training distribution ●.

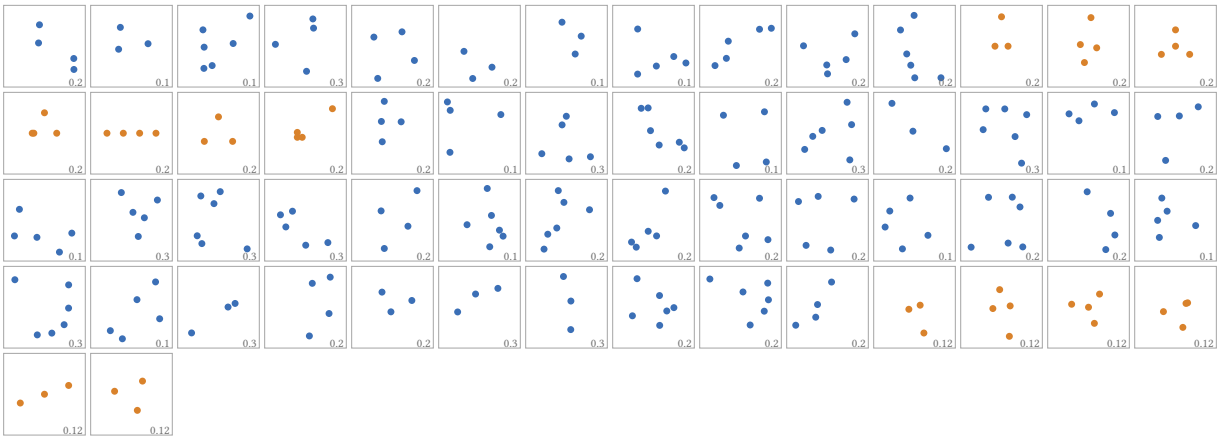


Figure 3: The wide set: the candidates of every configuration in the unit square and its deviation; random ●, hand-made ●, out of the training distribution ●.

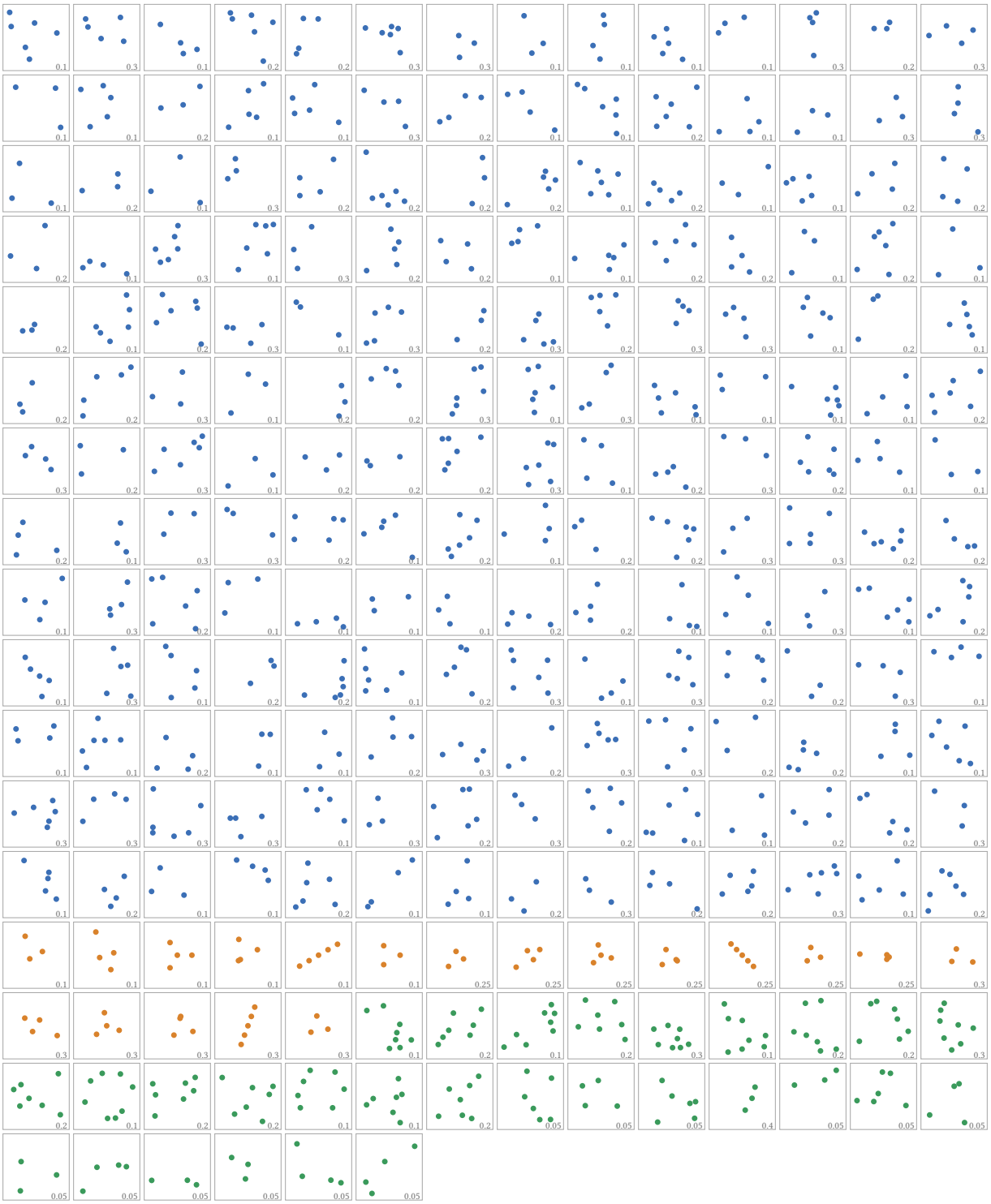


Figure 4: The test set: the candidates of every configuration in the unit square and its deviation; random ●, hand-made ●, out of the training distribution ●.

4. Making the scoring fast

Scoring a method runs its blocks on the voters of every configuration, sincerely and buried at every pair of frontrunners (about five a configuration): a few hundred small numpy calls on a grid of 324 points, where the calls cost more than the arithmetic. What the search does about it:

- The shares of the voters every method of the grammar can read are computed once for every configuration and kept (`Prepared`), each method getting only those it reads; what burial takes from a score ballot is remembered too (`MemoGrid`).

- What every subtree that elects or chooses candidates gives is remembered for each configuration and pair of frontrunners (`Memo`, 256 MB a process, the least recently used going first): a child of the genetic programming keeps most of its parent, an enumeration most of the smaller methods. A block runs its children on the voters it got, so a subtree gives the same wherever it is. About half of the work of a genetic programming is so saved: 214 ms a method became 105 ms on 18 configurations, sincerely and buried, with the same errors to the last digit, and the maps of the library of Section 6 are 2–4 ms a method and configuration.
- The compiled kernels run in the worker's own thread, not handed to a thread pool chunk by chunk.
- The workers (4 by default, a process each) score methods in parallel, each with a time limit (60 s).

5. Genetic programming

`yee-search gp` evolves a population of 1000 trees of the grammar. The first population is the methods of `yeelab.build.methods` the grammar can build, the best of a search of Conditionals where given (`--seed-from`, 20 of each of its rankings) and random trees of depth 3 to 8. Each generation keeps the 5 best, adds 25 % of new random trees, and fills the rest with children of parents chosen by tournaments of 5: a crossover swaps subtrees of the same type (half of the children), a mutation regrows a subtree, redraws one node or hoists a subtree into the place of its ancestor. Of the methods that err alike on every configuration only the best stays, and where the best has not improved for 30 generations the whole population starts again from random trees. The 50 best methods of the training (the hall of fame) are scored on the validation set after every generation. A run writes every method scored (`evals.jsonl`), so that a run stopped (by a power cut: `resume.sh`) goes on without scoring any method twice.

6. The search of Conditionals

`yee-search conditional` scores every Conditional of two methods of a library, and of two conditions and three methods, exactly, without running a block for any of them.

6.1. The library and its maps

The library is every method of the grammar of at most 8 blocks (enumerated, not drawn), the methods of `yeelab.build.methods` the grammar can build and the best of the GP runs. For each one and each configuration its **map** is kept: the winner at every point, sincerely and buried (`maps/`, shared by every run). Methods with the same maps everywhere are one: of 8712 methods 818 differ on `wide`.

6.2. Why the maps are enough

Burial is pointwise: the poll names the frontrunners at a point from the sincere first choices, whatever the method, and the method then runs on the buried voters of the point. So a Conditional buried elects where its condition is true buried what its first branch elects buried, else what its second does,

$$\mathrm{Burial}(\mathrm{Conditional}(c, A, B)) = \mathrm{where}(c_{\mathrm{buried}}, A_{\mathrm{buried}}, B_{\mathrm{buried}}), \tag{7}$$

and a condition $X == Y$ is where the maps of X and Y agree. The maps of the library give the maps of every Conditional of it; the tests check them against the blocks run.

6.3. Every Conditional at once

The loss of a point depends only on its winner, so with l_X the losses of method X at every point (of every configuration, sincere and buried, weighted as in J),

$$J(\mathrm{Conditional}(c, A, B)) = c \cdot l_A + (1 - c) \cdot l_B + \mathrm{cost}, \tag{8}$$

a part of A alone and a part of B alone. For a condition, the best two methods are the best A of the first part and the best B of the second, and the products $c \cdot l_X$ of all conditions with all methods are one matrix product: about $1.5 \cdot 10^{12}$ Conditionals in 20 minutes, under a nanosecond each. The conditions are where a method elects someone, `Top` or `Lead` of a tally against every threshold, $X == Y$ of every two methods of the library (about 300 000 that differ), and `&` of two of the best of those, each maybe negated: with A and B swapped, every `&` and `|`.

Two levels separate as well:

$$J(\text{Conditional}(c_1, A_1, \text{Conditional}(c_2, A_2, B))) = c_1 \cdot l_{A_1} + (\neg c_1 \wedge c_2) \cdot l_{A_2} + (\neg c_1 \wedge \neg c_2) \cdot l_B + \text{cost}, \tag{9}$$

three parts of one method each, so one matrix product a pair of conditions gives the best three methods. Every two of about 1900 conditions are tried: the best of all, and the best agreements $X == Y$ of FPTP, the two-round system and D20 with any other method.

6.4. Exact scores

The matrix products sum over the points, which the trimmed mean and the complexity do not: the products leave out the 5 % easiest and hardest configurations for all methods alike (by the median error of the library, as a method's best and worst configurations are mostly those) and pay for every block of each part. The best of them, by the products, are then scored exactly. They are ranked five ways: by J, without the cost of the complexity ("raw", to see whether a Conditional helps at all), the methods of the library alone, and two levels by J and raw.

7. The plain vocabulary

With `--plain` both searches use only methods that are simple to explain, each one a leaf: FPTP, Two Round, the Condorcet winner, DAC, D20, MJ, the trimmed means of the six scores and of Coarse(…, levels=3) at a trim of 0, 5, …, 45 or 49 (a trim of 0 is Score), and Coarse(…, levels=2), whose trim changes nothing: of two scores the trimmed mean orders the candidates as the mean does. Only a Conditional joins them, at most two inside each other (`grammar.PLAIN_NESTING`). Its condition is a comparison of two methods, whether a method elects someone, Top or Lead of the first choices or of the mean score against a threshold, or those combined. The Condorcet winner elects no one in a cycle, so it needs such a Conditional around it, a cycle being as bad as always.

The hard rule of the rivals is left out, on the set `free`: on the web UI's two setups no plain method comes near `search_6`, so the rule would be all that J measures. The setups still weigh three configurations each.

8. The coarse vocabulary

With `--coarse` both searches use the six scores counted in fewer groups, `Coarse(..., levels=n)` for n = 2, 3 or 6 (6 being the six scores themselves), and the methods burial moves little. The leaves are FPTP, Two Round, the Condorcet winner, the mean of Coarse 2 and the trimmed means of Coarse 3 and 6 at a trim of 0, 5, …, 45 or 49; with `--extra` D20, DAC and KOTH too. A runoff (`coarse_runoff`) is the Condorcet winner of the score comparisons of a coarse ballot, equal scores abstaining or ranked (tie weight 0 or 1), among the candidates who draw at the top of a trimmed mean (`Drawn`), the two or three best of one, or the winners of two leaves. The ballot of the runoff need not be that of the trim: Coarse 2 at a trim of 20 % draws the candidates fewer than a fifth of the voters disapprove of, and Coarse 6 decides among them. A runoff is never among the candidates of another, nor a Conditional.

Conditionals join them, at most two inside each other, as in Section 7. A condition compares two methods, asks whether one elects someone, or is `has_winner`: some trim of Coarse 3 or 6 elects the winner of a method (`CoarseTrim(...).has_winner(method)`, computed exactly over all trims, a microsecond or a few a point); and those combined by & and |, `has_winner` never negated inside them, as no block negates it. So `Conditional(has_winner(3, two_round), two_round, X)` keeps the two-round winner where a trimmed mean of the three groups agrees with it. Top and Lead are not in it. The search of Conditionals takes these leaves and the runoffs among a draw or the best of a trimmed mean as its library (not those among two leaves, left to the genetic programming), and `has_winner` of each leaf as conditions, from maps of their own that are no primitives (`primitives.Holds`). The rivals are left out (the set `free`), as in Section 7. The genetic programming remembers conditions as it remembers methods (`evaluate.Memo`), so a child that keeps its parent's `has_winner` does not compute it again.

9. Choosing the methods

Of millions of methods, the best on a few configurations is partly chance. So the best 200 of each ranking on the training set are scored on the validation set, the best 20 of those on the test set, and only the test set, of other configurations and a finer grid, tells how good a method is. The test set has been used to choose the methods of the web UI's rows, which makes their test scores somewhat too good; a new set of the same kind, used once, tells how much. Two methods are best compared configuration by configuration: the table of Section 11 gives on how many test configurations a method buries better than Star Two Round and on how many worse.

10. Running it

```
cd search
uv run yee-search conditional --out runs/cond --maps runs/2026-10-27-conditional/maps
uv run yee-search gp --out runs/night --until 06:00 --seed-from runs/cond
uv run yee-search report runs/night        # its best and the known methods on the test set
./night.sh 06:00 4                         # all three, until 06:00, with 4 workers
./night.sh 06:00 4 plain                   # the same over the plain vocabulary (@ch-plain)
./night.sh 06:00 4 coarse                  # the coarse vocabulary (@ch-coarse); coarse-extra with D20,
DAC, KOTH
```

A run of the search of Conditionals writes `library.json`, `train.json`, `validation.json`, `test.json` and `summary.md` (the table of Section 11); a GP run `evals.jsonl`, `validation.jsonl`, `generations.jsonl`, `hall.json` and the report. `night.sh` keeps the computer from sleeping while it runs, but not with the lid closed in KDE. Each GP worker holds about 2.5 GB of voters and remembered subtrees.

11. The methods found

method	J	J without cost	sincere	burial	burial better / worse than STR	complexity
`search_6`	0.0104	0.0087	0.0070	0.0093	58 % / 27 %	17
`search_7`	0.0107	0.0100	0.0065	0.0112	47 % / 43 %	7
`search_2`	0.0108	0.0101	0.0063	0.0114	45 % / 47 %	7
`search_1`	0.0108	0.0100	0.0063	0.0113	45 % / 46 %	8
`search_5`	0.0109	0.0087	0.0065	0.0094	61 % / 23 %	22
`search_3`	0.0122	0.0114	0.0079	0.0125	46 % / 47 %	8
`search_4`	0.0127	0.0121	0.0102	0.0127	37 % / 50 %	6
`star`	0.0128	0.0123	0.0090	0.0134	33 % / 56 %	5
`star_two_round`	0.0142	0.0131	0.0126	0.0133	0 % / 0 %	11
`mj`	0.0240	0.0238	0.0260	0.0231	23 % / 75 %	2
`two_round`	0.0524	0.0518	0.0518	0.0518	17 % / 78 %	6
`irv`	0.0556	0.0554	0.0529	0.0562	11 % / 84 %	2
`fptp`	0.1054	0.1052	0.1052	0.1052	4 % / 95 %	2

Table 4: The methods of the web UI's rows on the test set (230 configurations, grid 32): sincere and burial are the means over the configurations J keeps.

The three found last, as `yeelab.build.methods` writes them (`nanson`, `d20`, `mj`, `M` and `score_ballot` are its names):

```
search_5 = Conditional(
    nanson == Highest(Tally(score_ballot), among=Highest(Tally(Plurality()), n=2)),
    when_true=nanson,
    when_false=Conditional(Unbeaten(ScoreComparisons(score_ballot)) == d20,
        when_true=d20,
        when_false=Highest(Tally(score_ballot, statistic="median"), among=Highest(Weakest(M), n=2))))
search_6 = Conditional(
    Highest(Tally(score_ballot), among=Highest(Tally(Plurality()), n=2)) == nanson,
    when_true=nanson,
```

```
    when_false=Unbeaten(ScoreComparisons(score_ballot),
        among=Highest(Tally(score_ballot, statistic="median"),
                      among=Highest(Tally(score_ballot), n=2)) | d20))
search_7 = Unbeaten(ScoreComparisons(score_ballot, tie_weight=1.0),
                    among=mj | Unbeaten(ScoreComparisons(score_ballot)))
```

`search_6`: Nanson's winner where the higher mean score of the two with the most first choices agrees with it, else a runoff on the scores of each ballot between the higher median score of the two best by mean score and D20′s winner. `search_7`: the runoff of `search_1` between the majority judgment and the Condorcet winner of the score comparisons, counting each ballot's ranking where its scores tie.